Assignment1 solution

1. Priority encoder

- **Design**

  The priority encoder design has two problems, syntax error (in the output port Y definition, type reg is missing) and a bug in the reset operation as the reset signal resets only the output Y and keeps the valid bit unchanged.

```
module priority_enc (
input  clk,
input  rst,
input  [3:0] D,
output [1:0] Y, //! error must be of type reg
output reg valid
);

always @(posedge clk) begin
  if (rst)
      Y <= 2'b0; //! valid bit is not reset    Illegal reference to net "Y".
  else
    casex (D)
        4'b1000: Y <= 0;    Illegal reference to net "Y".
        4'bX100: Y <= 1;    Illegal reference to net "Y".
        4'bXX10: Y <= 2;    Illegal reference to net "Y".
        4'bXXX1: Y <= 3;    Illegal reference to net "Y".
    endcase
    valid <= (~|D)? 1'b0: 1'b1;
end
endmodule
```

design before justifications

```
module priority_enc (
    input clk,
    input rst,
    input [3:0] D,
    output reg [1:0] Y,
    output reg valid
);

  always @(posedge clk) begin
    if (rst) begin
      Y <= 2'b0;
      valid <= 1'b0;
    end else begin
      casex (D)
        4'b1000: Y <= 0;
        4'bX100: Y <= 1;
        4'bXX10: Y <= 2;
        4'bXXX1: Y <= 3;
      endcase
      valid <= (~|D) ? 1'b0 : 1'b1;
    end
  end
endmodule
```

design after justifications

- **Verification plan**

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ENCODER_1	When the reset is asserted, the outputs Y and valid must be 0	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ENCODER_2	exhaustively verify all possible inputs to check the encoder normal operation	Directed during the simulation	-	A checker in the testbench to make sure the output is correct for each value of D input

- **Testbench**

```
module priority_enc_tb ();
  //?parameters
  parameter CLK_CYCLE = 2;
  //?interface ports
  logic [3:0] D;
  logic clk;
  logic rst;
  logic [1:0] Y;
  logic valid;
  //?variables
  int correct_count, error_count = 0;
  //?instantiation part
  priority_enc PE_DUT (
      .D(D),
      .clk(clk),
      .rst(rst),
      .Y(Y),
      .valid(valid)
  );
  //?CLK generator
  always #(CLK_CYCLE / 2) clk = ~clk;
  //?test block
  initial begin
    clk = 0;
    D   = 0;
    rst = 0;
    #(CLK_CYCLE / 2);
    assert_reset();
    assert_data();
    assert_reset();
    $display("error_counter=%0d, correct_counter=%0d", error_count, correct_count);
    $stop;
  end
  //?tasks
  task assert_reset();
    rst = 1;
    #(CLK_CYCLE);
    test_result();
    rst = 0;
  endtask

  task assert_data();
    for (int i = 0; i < 17; i++) begin
      test_result();
      D++;
      #CLK_CYCLE;
    end
  endtask
```

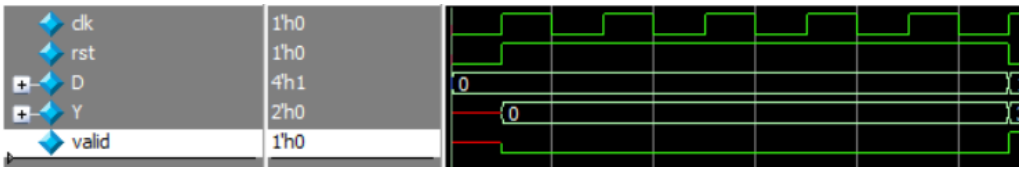
```sv
task test_result();
  //*check for reset
  if (rst == 1) begin
    check_result(0, 0, 0);
  end  //* check for the valid signal when all inputs are set to 0
  else if (D == 0) begin
    check_result(0, 0, 0);
  end  //* check for outputs at different input stimulus
  else if (D[0] == 1) begin
    check_result(1, 3, 0);
  end else if (D[1] == 1) begin
    check_result(1, 2, 1);
  end else if (D[2] == 1) begin
    check_result(1, 1, 2);
  end else if (D[3] == 1) begin
    check_result(1, 0, 3);
  end
endtask

task check_result(valid_exp, logic [1:0] Y_exp, int D_num);
  if (rst == 1) begin
    if (valid == valid_exp && Y == Y_exp) begin
      $display("Reset test passed");
      correct_count++;
    end else begin
      $display("Reset test fails");
      $display(
          "valid_exp=%0b, Y0_exp=%0b, Y1_exp=%0b ... valid_result=%0b, Y0_result=%0b, Y1_result=%0b",
          valid_exp, Y[0], Y[1], valid, Y[0], Y[1]);
      error_count++;
    end
  end else if (D_num == 0) begin
    if (valid == valid_exp) begin
      correct_count++;
      $display("valid = 0 when all inputs are set to 0 passed");
    end else begin
      error_count++;
      $display("valid = 0 when all inputs are 0 fails");
      $display("valid_exp=%0d ... valid_result=%0b", valid_exp, valid);
    end
  end else if (valid == valid_exp && Y == Y_exp) begin
    correct_count++;
    $display("outputs when D%0d is set to one are true (test passed)", D_num);
  end else begin
    error_count++;
    $display("outputs when D%0d is set to one are false (test fails)", D_num);
    $display(
        "valid_exp=%0b, Y0_exp=%0b, Y1_exp=%0b ... valid_result=%0b, Y0_result=%0b, Y1_result=%0b",
        valid_exp, Y_exp[0], Y_exp[1], valid, Y[0], Y[1]);
  end
endtask
endmodule
```

- **DO file**

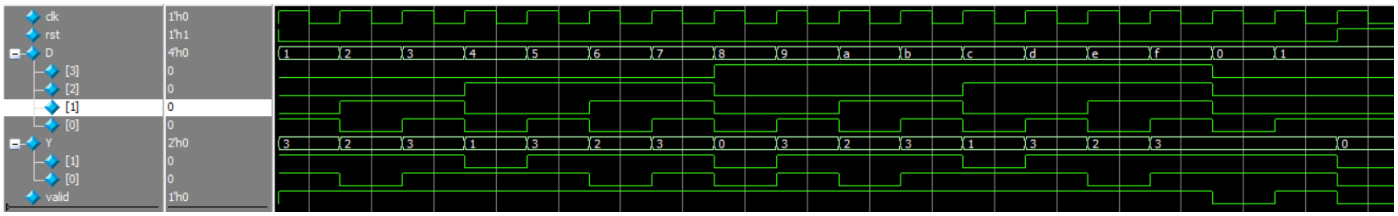
```
vlib work
vlog priority_enc.v priority_enc_tb.sv  +cover -covercells
vsim -voptargs=+acc work.priority_enc_tb -cover
coverage save priority_enc_tb.ucdb -onexit -du priority_enc
run -all
quit -sim
vcover report priority_enc_tb.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

❖ **Encoder_1 test case:**



❖ **Encoder_2 test case:**



- ## Coverage report

❖ Branch coverage

```
Coverage Report by instance with details

================================================================================
=== Instance: /\priority_enc_tb#PE_DUT
=== Design Unit: work.priority_enc
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         7         7         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\priority_enc_tb#PE_DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File priority_enc.v
-----------------------------------IF Branch-----------------------------------
    10                                        19     Count coming in to IF
    10             1                           2       if (rst)

    12             1                          17       else

Branch totals: 2 hits of 2 branches = 100.00%
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       7         7         0   100.00%

================================Statement Details================================
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         18        18         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\priority_enc_tb#PE_DUT  --

                                   Node      1H->0L      0L->1H  "Coverage"
                                   ----------------------------------------
                                 D[0-3]           1           1      100.00
                                 Y[1-0]           1           1      100.00
                                    clk           1           1      100.00
                                    rst           1           1      100.00
                                  valid           1           1      100.00

Total Node Count     =          9
Toggled Node Count   =          9
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (18 of 18 bins)


Total Coverage By Instance (filtered view): 100.00%
```

2. ALU

- **Design**

  Design is ok. No bugs reported. The default line will be excluded from branch and statement coverage to achieve 100% coverage.

```
module ALU_4_bit (
    input  clk,
    input  reset,
    input  [1:0] Opcode,   // The opcode
    input  signed [3:0] A, // Input data A in 2's complement
    input  signed [3:0] B, // Input data B in 2's complement

    output reg signed [4:0] C // ALU output in 2's complement

        );

   reg signed [4:0]      Alu_out; // ALU output in 2's complement

   localparam         Add             = 2'b00; // A + B
   localparam         Sub             = 2'b01; // A - B
   localparam         Not_A            = 2'b10; // ~A
   localparam         ReductionOR_B  = 2'b11; // |B

   // Do the operation
   always @* begin
      case (Opcode)
         Add:            Alu_out = A + B;
         Sub:            Alu_out = A - B;
         Not_A:          Alu_out = ~A;
         ReductionOR_B:  Alu_out = |B;
         default:  Alu_out = 5'b0;  //! execlude this line from branch and statement coverage
      endcase
   end // always @ *

   // Register output C
   always @(posedge clk or posedge reset) begin
      if (reset)
        C <= 5'b0;
      else
        C<= Alu_out;
   end

endmodule
```

- **Verification plan**

	Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
4	ALU_1	when the reset is asserted, the output C is set to 0	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
5	ALU_2	check all compinations of the possible corner values for A&B in both addition and subtraction (max_positive, max_negative, zero, add, sub) which will result in 18 different test cases	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
6	ALU_3	set B to a random value and try A with all ones, all zeros and walking ones then test the bitwise not operation	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
7	ALU_4	set A to a random value and try B with all ones, all zeros and walking ones then test the reduction or operation	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
8	ALU_5	try to randomize everything A,B and opcode for a while and test the output for any change in the input values	Randomization	-	A checker in the testbench to make sure the output is correct

- **Testbench**

```
module ALU_tb ();
  //? local parameters
  localparam MAXPOS = 7;
  localparam MAXNEG = -8;
  localparam Add = 2'b00;
  localparam Sub = 2'b01;
  localparam Not_A = 2'b10;
  localparam ReductionOR_B = 2'b11;
  localparam CLK_CYCLE = 2;
  //? Interface signals
  logic reset, clk;
  logic signed [3:0] A, B;
  logic signed [4:0] C;
  logic [1:0] opcode;
  //? Variables
  int error_count, correct_count;
  //? DUT instantiation
  ALU_4_bit DUT_ALU (
      .clk(clk),
      .reset(reset),
      .Opcode(opcode),
      .A(A),
      .B(B),
      .C(C)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? Driver block
  initial begin
    //! ALU_1 test
    clk = 0;
    reset = 0;
    A = $random;
    B = $random;
    opcode = $random;
    #(CLK_CYCLE) assert_reset();
```

```
    //! ALU_2 test
    //op-code iterator
    for (int i = 0; i < 2; i++) begin
      if (i == 0) opcode = Add;
      else opcode = Sub;
      //input A iterator
      for (int j = 0; j < 3; j++) begin
        if (j == 0) A = 0;
        else if (j == 1) A = MAXPOS;
        else A = MAXNEG;
        //input B iterator
        for (int k = 0; k < 3; k++) begin
          if (k == 0) B = 0;
          else if (k == 1) B = MAXPOS;
          else B = MAXNEG;
          if (opcode == Add) check_result(A + B);
          else check_result(A - B);
        end
      end
    end
    //! ALU_3 test
    B = $random;
    A = 4'b1111;
    opcode = Not_A;
    check_result(~A);
    A = 4'b0000;
    check_result(~A);
    A = 4'b0001;
    check_result(~A);
    A = 4'b0010;
    check_result(~A);
    A = 4'b0100;
    check_result(~A);
    A = 4'b1000;
    check_result(~A);
    //! ALU_4 test
    A = $random;
    B = 4'b1111;
    opcode = ReductionOR_B;
    check_result(|B);
    B = 4'b0000;
    check_result(|B);
    B = 4'b0001;
    check_result(|B);
    B = 4'b0010;
    check_result(|B);
    B = 4'b0100;
    check_result(|B);
    B = 4'b1000;
    check_result(|B);
```

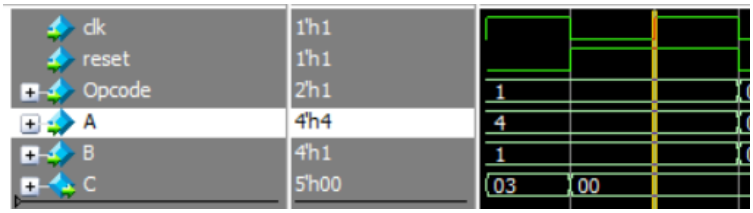
```
    //! ALU_5 test
    for(int i=0; i<100; i++)begin
      A = $random;
      B = $random;
      opcode = $random;
      if(i<10) begin
        reset = 1;
        check_result(0);
      end
      else begin
        reset = 0;
        if(opcode == Add) check_result(A+B);
        else if(opcode == Sub) check_result(A-B);
        else if(opcode == Not_A) check_result(~A);
        else if(opcode == ReductionOR_B) check_result(|B);
      end
    end
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task assert_reset();
    reset = 1;
    check_result(0);
    reset = 0;
  endtask

  task check_result(input [4:0] expected_C);
    #CLK_CYCLE;
    if (expected_C == C) correct_count++;
    else begin
      error_count++;
      $display("xxxx ERROR: the actual output result is not matching the expected result xxxx");
      $display("A=%0b ,B=%0b, C_exp=%0b, C_actual=%0b, opcode=%0b, reset=%0b", A, B, expected_C, C, opcode, reset);
    end
  endtask

endmodule
```

- **DO file**

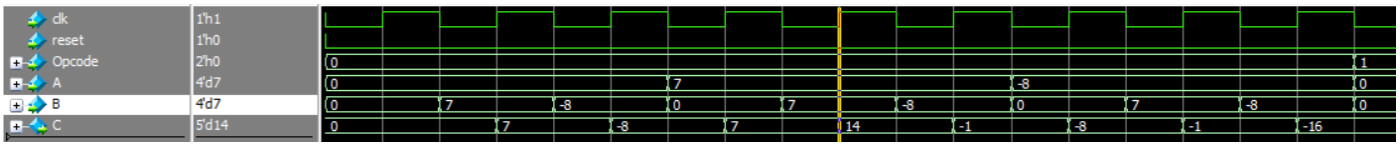
```
vlib work
vlog ALU.v ALU_tb.sv  +cover -covercells
vsim -voptargs=+acc work.ALU_tb -cover
coverage save ALU_tb.ucdb -onexit -du ALU_4_bit
#do wave.do
#log -r /*
run -all
coverage exclude -src ALU.v -line 26 -code s
coverage exclude -src ALU.v -line 26 -code b
quit -sim
vcover report ALU_tb.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

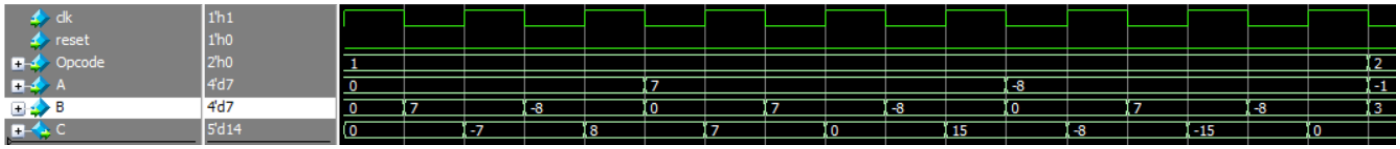
❖ **ALU_1 test**



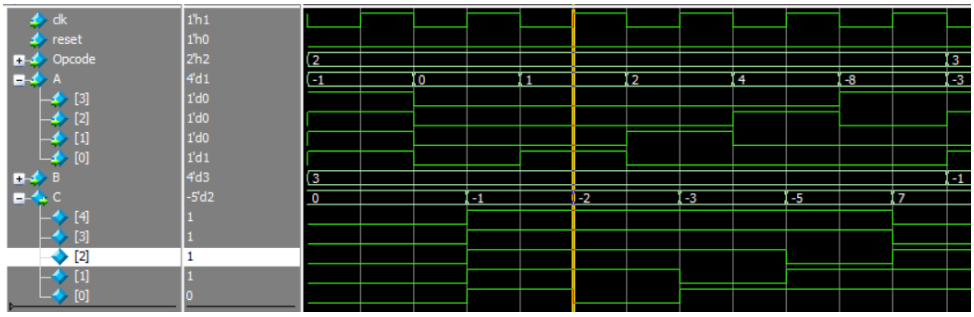
❖ **ALU_2 test (addition)**



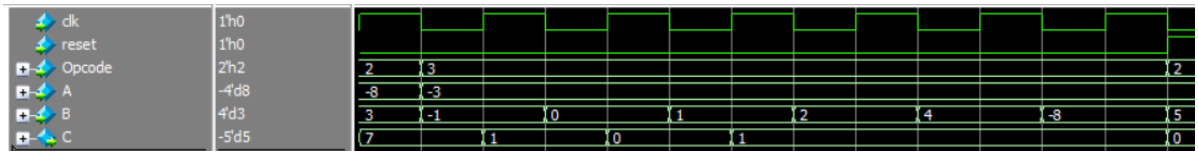
❖ **ALU_2 test (subtraction)**



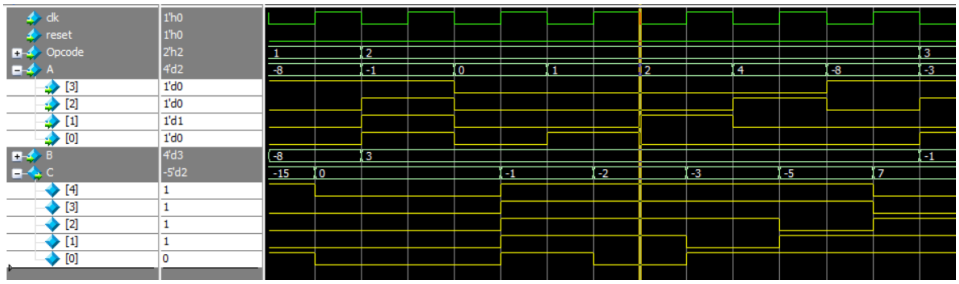
❖ **ALU_2 test (bitwise not)**



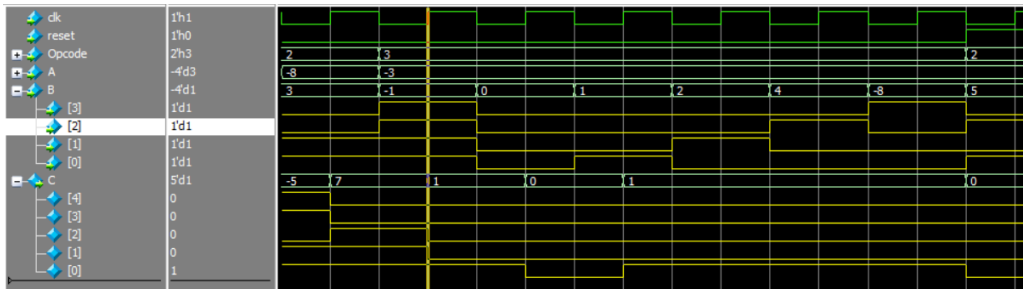
❖ **ALU_2 test (reduction or)**



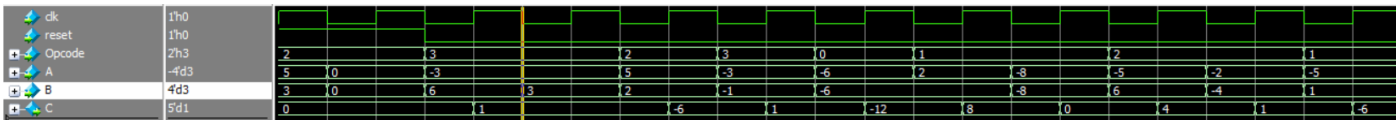
❖ **ALU_3 test**



❖ **ALU_4 test**

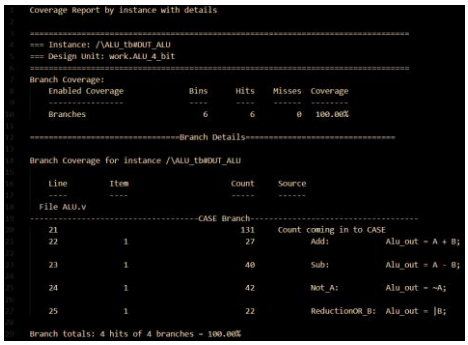


❖ **ALU_5 test**



- **<u>Coverage report</u>**

❖ **Branch coverage**



```
Coverage Report by instance with details

================================================================================
=== Instance: /\ALU_tb#DUT_ALU
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         6         6         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\ALU_tb#DUT_ALU

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALU.v
------------------------------------CASE Branch------------------------------------
    21                                       131     Count coming in to CASE
    22             1                          27           Add:            Alu_out = A + B;

    23             1                          40           Sub:            Alu_out = A - B;

    24             1                          42           Not_A:          Alu_out = ~A;

    25             1                          22           ReductionOR_B:  Alu_out = |B;

Branch totals: 4 hits of 4 branches = 100.00%
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%

================================Statement Details================================
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         44        44         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\ALU_tb#DUT_ALU  --

                                      Node      1H->0L      0L->1H  "Coverage"
                                      ----------------------------------------
                                     A[0-3]           1           1      100.00
                                Alu_out[4-0]          1           1      100.00
                                     B[0-3]           1           1      100.00
                                     C[4-0]           1           1      100.00
                                  Opcode[0-1]         1           1      100.00
                                       clk            1           1      100.00
                                     reset            1           1      100.00

Total Node Count     =         22
Toggled Node Count   =         22
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%
```

3. DSP

- **Design**

  The design has two bugs, C_reg doesn't have a reset signal, there is an extra stage in the adder pipeline path which is incorrect and will cause an extra delay cycle.

```
module DSP(A, B, C, D, clk, rst_n, P);
parameter OPERATION = "ADD";
input  [17:0] A, B, D;
input  [47:0] C;
input clk, rst_n;
output reg  [47:0] P;

reg  [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg, adder_out_stg1, adder_out_stg2;
reg  [47:0] C_reg, mult_out;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // reset
         A_reg_stg1 <= 0;
         A_reg_stg2 <= 0;
         B_reg <= 0;
         D_reg <= 0;
         adder_out_stg1 <= 0;
         mult_out <= 0;
         P <= 0;
         //! C_reg reset is missing from this block
    end
    else begin
        A_reg_stg1 <= A;
        A_reg_stg2 <= A_reg_stg1;
        B_reg <= B;
        C_reg <= C;
        D_reg <= D;
        adder_out_stg2 <= adder_out_stg1; //! adder_out_stg2 is an extra delay stage
        if (OPERATION == "ADD") begin
            adder_out_stg1 <= D_reg + B_reg;
            P <= mult_out + C_reg;
        end
        else if (OPERATION == "SUBTRACT") begin
            adder_out_stg1 <= D_reg - B_reg;
            P <= mult_out - C_reg;
        end
        mult_out <= A_reg_stg2 * adder_out_stg2;
    end
end

endmodule
```

```
module DSP(A, B, C, D, clk, rst_n, P);
parameter OPERATION = "ADD";
input  [17:0] A, B, D;
input  [47:0] C;
input clk, rst_n;
output reg  [47:0] P;

reg  [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg, adder_out;
reg  [47:0] C_reg, mult_out;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // reset
         A_reg_stg1 <= 0;
         A_reg_stg2 <= 0;
         B_reg <= 0;
         D_reg <= 0;
         adder_out <= 0;
         mult_out <= 0;
         C_reg <= 0;
         P <= 0;
    end
    else begin
        A_reg_stg1 <= A;
        A_reg_stg2 <= A_reg_stg1;
        B_reg <= B;
        C_reg <= C;
        D_reg <= D;
        if (OPERATION == "ADD") begin
            adder_out <= D_reg + B_reg;
            P <= mult_out + C_reg;
        end
        else if (OPERATION == "SUBTRACT") begin
            adder_out <= D_reg - B_reg;
            P <= mult_out - C_reg;
        end
        mult_out <= A_reg_stg2 * adder_out;
    end
end

endmodule
```

- ## Verification plan

1	Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
9	DSP_1	When the reset is asserted, the output P must be 0	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
10	DSP_2	try to randomize input signals A,B,C,D and see the output after 4clk cycles	Randomization	-	A checker in the testbench to make sure the output is correct

- ## Testbench

```
module DSP_tb ();
  //? local parameters
  localparam  CLK_CYCLE = 2;
  //? Interface signals
  logic signed [17:0] A,B,D;
  logic [47:0] C,P;
  logic clk,rst_n;
  //? Temp signals
  logic [17:0] adder_out_exp;
  logic [47:0] mulout_exp, P_exp;
  //? Variables
  int error_count, correct_count;
  //? DUT instantiation
  DSP DUT_DSP (
    .clk(clk),
    .rst_n(rst_n),
    .A(A),
    .B(B),
    .C(C),
    .D(D),
    .P(P)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? Driver block
  initial begin
    A = 0;
    B = 0;
    C = 0;
    D = 0;
    rst_n = 1;
    clk = 0;
    #CLK_CYCLE;
    //! DSP_1 test
    assert_reset();
    //! DSP_2 test
    for(int i=0; i<100; i++)begin
      A = $urandom;
      B = $urandom;
      C = {$urandom, $urandom};
      D = $urandom;
      adder_out_exp = (B+D);
      mulout_exp = adder_out_exp * A;
      P_exp = mulout_exp + C;
      check_result(P_exp);
    end
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
```

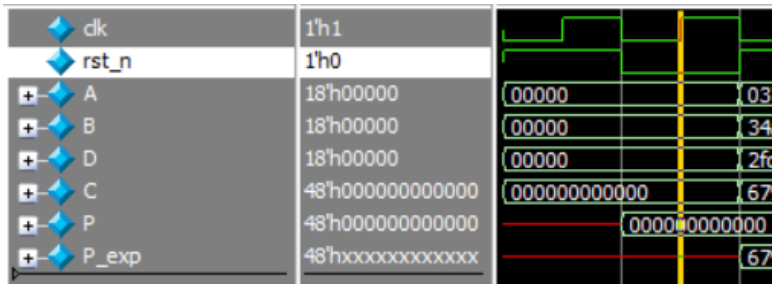
```
  //? TASKS
  task assert_reset();
    rst_n = 0;
    check_result(0);
    rst_n = 1;
  endtask

  task check_result(input [47:0] expected_P);
    if(rst_n == 0) #CLK_CYCLE;
    else #(4*CLK_CYCLE);

    if (expected_P == P) correct_count++;
    else begin
      error_count++;
      $display("xxxx ERROR: the actual output result is not matching the expected result xxxx");
      $display("A=%0h ,B=%0h, C=%0h, D=%0h, P_exp=%0h, P_result=%0h", A, B, C, D, expected_P, P);
    end
  endtask

endmodule
```

- **DO file**

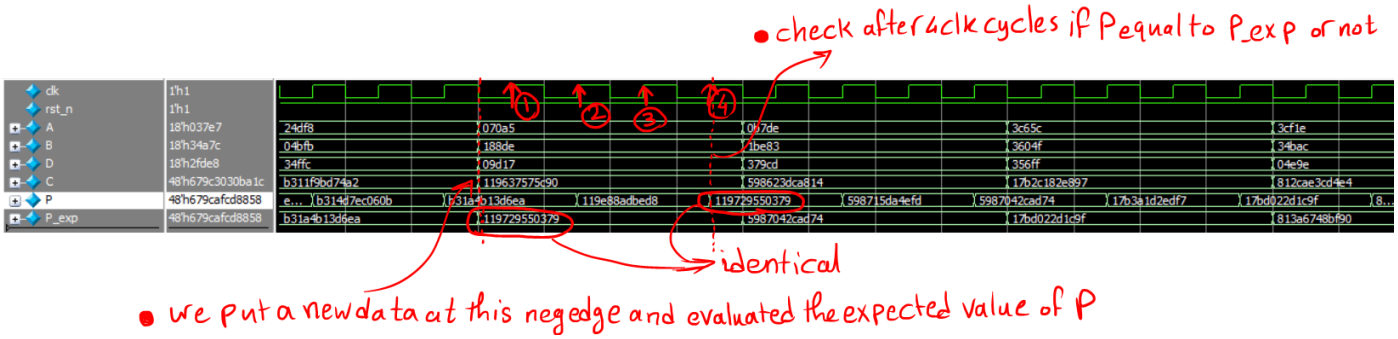
```
vlib work
vlog DSP.v DSP_tb.sv  +cover -covercells
vsim -voptargs=+acc work.DSP_tb -cover
coverage save DSP_tb.ucdb -onexit -du DSP
do wave.do
log -r /*
run -all
coverage exclude -du DSP -togglenode {mult_out[36]}
coverage exclude -du DSP -togglenode {mult_out[37]}
coverage exclude -du DSP -togglenode {mult_out[38]}
coverage exclude -du DSP -togglenode {mult_out[39]}
coverage exclude -du DSP -togglenode {mult_out[40]}
coverage exclude -du DSP -togglenode {mult_out[41]}
coverage exclude -du DSP -togglenode {mult_out[42]}
coverage exclude -du DSP -togglenode {mult_out[43]}
coverage exclude -du DSP -togglenode {mult_out[44]}
coverage exclude -du DSP -togglenode {mult_out[45]}
coverage exclude -du DSP -togglenode {mult_out[46]}
coverage exclude -du DSP -togglenode {mult_out[47]}
quit -sim
vcover report DSP_tb.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

  - **DSP_1 test**



  - **DSP_2 test**



- **Coverage report**

  - **branch coverage**

```
Coverage Report by instance with details

================================================================================
=== Instance: /\DSP_tb#DUT_DSP
=== Design Unit: work.DSP
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         2         2         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\DSP_tb#DUT_DSP

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File DSP.v
------------------------------------IF Branch------------------------------------
    12                                       403     Count coming in to IF
    12              1                          2       if (!rst_n) begin

    23              1                        401       else begin

Branch totals: 2 hits of 2 branches = 100.00%
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      17        17         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\DSP_tb#DUT_DSP  --
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        652       652         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\DSP_tb#DUT_DSP  --

                                    Node      1H->0L      0L->1H  "Coverage"
                                    ----------------------------------------
                                    A[0-17]        1           1      100.00
                             A_reg_stg1[17-0]      1           1      100.00
                             A_reg_stg2[17-0]      1           1      100.00
                                    B[0-17]        1           1      100.00
                                 B_reg[17-0]       1           1      100.00
                                    C[0-47]        1           1      100.00
                                 C_reg[47-0]       1           1      100.00
                                    D[0-17]        1           1      100.00
                                 D_reg[17-0]       1           1      100.00
                                    P[47-0]        1           1      100.00
                              adder_out[17-0]      1           1      100.00
                                      clk          1           1      100.00
                               mult_out[35-0]      1           1      100.00
                                      rst_n        1           1      100.00

Total Node Count     =        326
Toggled Node Count   =        326
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (652 of 652 bins)


Total Coverage By Instance (filtered view): 100.00%
```